

架构决策记录 ADR 集合

ADR-001：采用前后端分离的模块化分层单体架构

状态

已接受

背景

校园互助服务平台需要在 10 周课程周期内完成可演示版本，团队规模约 4 人。系统包含用户认证、任务发布接单、社区互动、评价仲裁、消息通知和后台管理等模块。候选方案包括分层单体、微服务和事件驱动架构。

决策

我们决定采用 **前后端分离 + 模块化分层单体架构**。前端使用 Vue 3，后端使用 Spring Boot，后端内部按 Controller、Service、Mapper、Entity / DTO 分层，并按业务模块组织代码。

理由

- 分层单体能满足校园级规模和 500 人高峰在线的性能要求。
- 开发、部署、调试和测试成本明显低于微服务。
- 后端模块边界清晰，后续可以将消息、支付、审核等模块拆分出去。
- 任务接单、完成、评价和仲裁需要同步事务一致性，分层单体更容易控制。
- 当前团队时间有限，微服务的服务治理和事件驱动的一致性处理会挤压业务开发时间。

后果

- 正面影响：架构简单、迭代快、调试方便，能在课程周期内优先完成核心业务。
- 负面影响：所有模块同进程部署，局部高负载模块不能单独扩容。
- 需要关注的风险：后端代码必须保持模块边界，否则单体会逐渐变成难维护的大泥球。

AI 辅助记录

- AI 初稿内容摘要：AI 建议使用“前后端分离 + MVC 架构”，认为课程项目不适合微服务。
- 人工修订内容：将 MVC 扩展为“模块化分层单体”，明确 Controller / Service / Mapper 职责，并加入后续服务拆分路径。
- 修订理由：AI 初稿只描述了展示层和控制层，没有充分覆盖后端业务模块边界和演进策略。

AI 初稿与人工修订 diff

- 系统采用前后端分离 MVC 架构，后端提供接口，前端负责页面展示。

+ 系统采用前后端分离的模块化分层单体架构，后端按 Controller、Service、Mapper、Entity / DTO 分层，并按认证、任务、社区、评价、消息、后台管理划分业务模块。
- 暂不考虑微服务，因为课程项目开发成本较高。

+ 暂不采用微服务，但在模块边界、接口契约和外部系统适配层上保留演进空间；当消息、支付或审核模块负载独立增长时，再考虑服务拆分。

ADR-002：采用柔性实名与匿名社区并存的身份架构

状态

已接受

背景

P1 调研显示，用户既希望平台能通过学号认证降低诈骗风险，也希望树洞和部分社区场景可以匿名表达。若全站强制实名展示，会削弱用户表达意愿；若全站匿名，又会影响交易安全和纠纷追溯。

决策

我们决定采用 **柔性实名 + 匿名社区** 的身份架构。用户注册和交易相关功能必须完成学号 / 邮箱认证；前台交易页面展示昵称和认证标识；树洞、部分社区话题和评论支持匿名展示；后台在权限受控情况下保留必要追溯能力。

理由

- 交易场景需要身份可信，避免恶意接单、欺诈和纠纷无法追责。
- 树洞和情绪表达场景需要匿名保护，否则用户不愿发布真实内容。
- 匿名只影响前台展示，不等于数据库无身份记录，能兼顾治理和隐私。
- 身份展示规则应由后端统一控制，前端不能自行决定是否暴露真实身份。

后果

- 正面影响：兼顾安全、隐私和社区活跃度。
- 负面影响：身份模型和权限控制更复杂，需要区分真实身份、昵称、匿名展示名。
- 需要关注的风险：管理员权限必须受控，避免匿名内容被随意反查。

AI 辅助记录

- AI 初稿内容摘要：AI 倾向建议“强制实名制”，认为所有用户行为都应绑定真实学号展示。
- 人工修订内容：改为交易强认证、社区可匿名、后台有限追溯的混合方案。
- 修订理由：AI 忽略了 P1 用户调研中对树洞匿名和隐私表达的强需求，也没有区分交易与社区场景的不同风险。

AI 初稿与人工修订 diff

- 所有用户必须完成实名，并在平台所有页面展示学号或真实姓名，以提高安全性。
- + 所有用户必须完成学号 / 邮箱认证，但前台默认展示昵称和认证标识；交易场景强调可信身份，树洞和部分社区场景允许匿名展示。
- 匿名功能不建议提供，因为会增加违规内容风险。
- + 匿名功能仅影响前台展示，后端仍保留受控追溯能力；配合内容审核、举报和管理员仲裁降低违规风险。

ADR-003：以 MySQL 为主数据库，Redis 作为缓存和辅助能力，首版不引入 MongoDB

状态

已接受

背景

P1 需求文档中曾提出 MySQL、Redis、MongoDB 组合。但根据当前业务，用户、任务、参与关系、评价、消息、反馈等数据结构关系明确，需要事务和索引。树洞和评论虽然文本内容较多，但首版规模可由 MySQL TEXT 字段支撑。

决策

我们决定首版以 **MySQL 作为唯一持久化主数据库**，Redis 作为缓存、限流、未读数和令牌辅助校验工具。首版不引入 MongoDB。

理由

- MySQL 能支撑核心交易状态流转和评价仲裁的一致性要求。
- 当前表结构已有 users、tasks、task_participants、messages、task_comments、task_reviews、feedback 等关系模型。
- 引入 MongoDB 会增加部署、数据同步、备份和查询一致性成本。
- Redis 只作为性能优化和临时状态，不作为核心事实来源，降低数据丢失风险。

后果

- 正面影响：数据模型统一，事务处理简单，部署复杂度低。
- 负面影响：全文搜索、复杂非结构化内容分析能力有限。
- 需要关注的风险：任务列表、评论和消息量增长后，需要补充索引、归档策略或搜索引擎。

AI 辅助记录

- AI 初稿内容摘要：AI 建议使用 MySQL 存储结构化交易数据，MongoDB 存储树洞和聊天内容，Redis 做缓存。
- 人工修订内容：删除首版 MongoDB，保留 MySQL + Redis。
- 修订理由：AI 倾向堆叠技术，忽略了课程项目部署和维护成本；当前非结构化数据量不足以证明 MongoDB 的必要性。

AI 初稿与人工修订 diff

- 数据库采用 MySQL + MongoDB + Redis。MySQL 存储交易数据，MongoDB 存储树洞、聊天等非结构化数据，Redis 做缓存。
- + 首版采用 MySQL + Redis。MySQL 存储用户、任务、评论、消息、评价、反馈等全部持久化数据；Redis 仅用于缓存、限流、未读数和令牌辅助校验。
- MongoDB 可提高非结构化内容扩展性。
- + MongoDB 暂缓引入；当树洞内容量、搜索和内容分析需求明显增长时，再评估引入文档数据库或搜索引擎。

ADR-004：采用 REST JSON + JWT 作为前后端接口与鉴权方案

状态

已接受

背景

系统需要支持 Web 端和移动端访问。前端和后端分离部署，接口需要清晰、易测试、易调试。用户登录后访问任务、评论、消息、管理接口时需要携带身份信息。

决策

我们决定前后端接口采用 **REST JSON**，鉴权采用 **JWT Bearer Token**。后端通过 Spring Security 统一校验令牌和角色权限，接口返回统一响应结构。

理由

- REST JSON 简单直观，适合课程项目和前后端协作。
- JWT 适合前后端分离，不依赖服务器 Session。
- Spring Security 可集中处理权限，避免每个 Controller 重复鉴权逻辑。
- 统一响应结构有利于前端错误处理和接口测试。

后果

- 正面影响：接口易调试，前后端并行开发成本低。
- 负面影响：JWT 一旦签发，在过期前天然不易主动失效，需要配合短有效期或 Redis 黑名单。
- 需要关注的风险：JWT secret 不能硬编码到生产环境；刷新令牌和退出登录需要单独设计。

AI 辅助记录

- AI 初稿内容摘要：AI 建议使用 REST API 和 JWT，简单说明适合前后端分离。
- 人工修订内容：补充 Spring Security 统一鉴权、角色权限、令牌过期、退出登录和刷新令牌风险。
- 修订理由：AI 初稿没有充分讨论 JWT 主动失效、管理员权限和敏感接口保护问题。

AI 初稿与人工修订 diff

- 前后端使用 REST API 通信，登录后返回 JWT，后续请求携带 token。
- + 前后端使用 REST JSON 通信，登录后返回 access token 和 refresh token；后端通过 Spring Security 统一解析 JWT、校验角色权限，并对管理接口进行额外授权控制。
- JWT 简单方便，适合前后端分离。
- + JWT 需要设置较短有效期，退出登录和封禁用户时通过 Redis 黑名单或 tokenVersion 机制处理主动失效。